

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	A. Lakshmi
Name	192373066

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I A. Lakshmi(192373066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Lakshmi*

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. Design an HDFS-Based Storage Solution for a Media Platform

Introduction

A media platform stores large amounts of videos, audio files, images, thumbnails, user-uploaded content, and application logs. Since media files are generally large, HDFS is suitable because it is designed for distributed storage of large datasets.

The main constraints are:

- Data availability must be above 99.9%.
- Replication factor cannot exceed 3.
- Storage growth must remain cost efficient.

Proposed HDFS Design

The proposed solution uses an HDFS cluster with:

- One Active NameNode.
- One Standby NameNode for high availability.
- Multiple DataNodes for storing actual media data.
- Replication factor of 3.
- Rack-aware replica placement.
- Large HDFS block sizes, such as 256 MB or 512 MB, for large media files.
- Storage monitoring and data lifecycle management.

Replication Factor

Set the HDFS replication factor to **3**, which is the maximum allowed by the constraint.

For every HDFS block, three copies are maintained on different DataNodes.

For example, if a video is divided into several blocks, each block has three replicas. If one DataNode fails, the other two copies are still available.

This provides fault tolerance without violating the replication constraint.

Rack Awareness

The three replicas should be distributed across different racks whenever possible.

This is important because a complete rack may become unavailable due to:

- Power failure.
- Network failure.

- Hardware failure.
- Maintenance.

If all three replicas are stored in the same rack, a rack failure could make the data unavailable. Rack-aware placement reduces this risk.

NameNode High Availability

The NameNode maintains HDFS metadata. A NameNode failure can prevent clients from accessing the filesystem even when the actual data is still present.

Therefore, use:

- Active NameNode.
- Standby NameNode.
- JournalNodes for maintaining edit information.
- Automatic failover.

If the Active NameNode fails, the Standby NameNode can take over.

This removes the NameNode as a single point of failure and helps achieve the required availability.

Cost-Efficient Storage Growth

Replication factor 3 means that three times the logical storage capacity is required for replicated data. Therefore, storage must be managed carefully.

Cost efficiency can be achieved by:

1. Using high-capacity DataNodes.
2. Using large HDFS blocks for large media files.
3. Compressing logs and metadata where appropriate.
4. Moving old or rarely accessed data to cheaper storage.
5. Removing unnecessary duplicate datasets.
6. Monitoring storage utilization.
7. Adding DataNodes gradually as the media collection grows instead of purchasing excessive capacity initially.

Availability

The combination of:

- Three replicas.
- Rack-aware placement.

- Active/Standby NameNodes.
- Automatic failover.
- DataNode health monitoring.

provides strong fault tolerance and supports an availability target above 99.9%.

Justification

Constraint	Design Decision
Availability >99.9%	NameNode HA + 3 replicas + rack awareness
Replication ≤ 3	Replication factor = 3
Cost efficiency	High-capacity nodes + large blocks + lifecycle management
Media workload	HDFS distributed storage
Failure tolerance	Replicas distributed across DataNodes/racks

2. MapReduce Workflow for Processing Logs Within 20 Minutes

Introduction

The system needs to process a large volume of log data within a **20-minute batch window**, while the number of cluster nodes is limited.

A MapReduce workflow can process the logs in parallel by dividing the input into multiple splits and assigning Mapper and Reducer tasks across the available nodes.

Proposed Job Design

The workflow consists of:

1. Input data stored in HDFS.
2. Input split creation.
3. Mapper tasks.
4. Combiner.
5. Shuffle and Sort.
6. Reducer tasks.
7. Final output stored in HDFS.

For example, the log data may contain:

- Timestamp.

- User ID.
- IP address.
- URL.
- HTTP status code.
- Response time.

The MapReduce job can calculate:

- Number of requests.
- Number of errors.
- Requests per user.
- Requests per URL.
- Average response time.

Mapper

The Mapper reads each log record and converts it into key-value pairs.

For example, if the requirement is to count errors, a Mapper could generate:

(500, 1)

for every HTTP 500 error.

Similarly, for counting requests per user:

(User101, 1)

can be generated for each request from User101.

Combiner

A Combiner should be used whenever the aggregation operation allows it.

Suppose a Mapper generates:

(User101, 1), (User101, 1), (User101, 1), (User101, 1)

The Combiner can locally convert these into:

(User101, 4)

This significantly reduces the amount of intermediate data transferred during the Shuffle phase.

This is especially useful when cluster nodes and network bandwidth are limited.

Shuffle and Sort

During the Shuffle phase, Hadoop transfers Mapper output to the appropriate Reducer.

The data is grouped by key.

For example:

(User101, 4)

(User101, 2)

(User101, 5)

will be grouped as:

User101 $\rightarrow$ [4, 2, 5]

The Sort phase organizes keys before they are processed by the Reducers.

Reducer

The Reducer combines the values for each key.

For example:

User101 $\rightarrow$ [4, 2, 5]

becomes:

User101 $\rightarrow$ 11

The final output is written back to HDFS.

Task Distribution

Since the number of nodes is limited, tasks should be distributed carefully.

Suppose there are four worker nodes. Input data is divided into multiple HDFS blocks, and Mapper tasks are assigned to process those blocks.

Hadoop should preferably execute the Mapper on the node containing the corresponding HDFS block. This is called **data locality**.

Data locality reduces network traffic because the data does not have to travel across the network before processing.

The Reducer tasks should also be distributed across available nodes so that no single node becomes a bottleneck.

Techniques to Meet the 20-Minute Requirement

1. Controlled Parallelism

Run multiple Mapper tasks simultaneously instead of processing the entire dataset sequentially.

2. Data Locality

Run tasks close to where the input data is stored.

3. Combiner

Reduce the amount of intermediate data transferred between nodes.

4. Compression

Compress Mapper output to reduce network transfer during Shuffle.

5. Balanced Reducers

Use an appropriate number of Reducers. Too few Reducers create bottlenecks, while too many increase task-management overhead.

6. Avoid Data Skew

If one key contains a huge percentage of the data, one Reducer may become overloaded. Partitioning should therefore distribute work evenly.

7. Efficient Input Format

Use an input format appropriate for large log files and avoid creating unnecessarily small files.

3. YARN Scheduler Policy for ETL, Reporting and Machine Learning

Introduction

The YARN cluster supports three different workloads:

- ETL.
- Reporting.
- Machine Learning.

These workloads have different resource requirements and priorities.

The scheduler must satisfy:

- Fairness.
- Higher priority for ETL.
- Recovery from node failures.

Proposed Scheduler

A **Capacity Scheduler** with separate queues is suitable.

Create three queues:

1. ETL Queue.
2. Reporting Queue.
3. Machine Learning Queue.

An example resource allocation is:

Queue	Capacity	Priority
ETL	50%	High
Reporting	25%	Medium
Machine Learning	25%	Normal

The exact percentages can be adjusted according to the organization's workload.

ETL Queue

ETL is given the highest priority because it may prepare data required by other applications.

The ETL queue receives a guaranteed portion of cluster resources.

When additional resources are available, ETL can temporarily use them according to scheduler rules.

However, ETL should not permanently consume the entire cluster because that would violate fairness.

Reporting Queue

Reporting jobs receive a guaranteed resource allocation.

This prevents large ETL or ML jobs from completely blocking business reports.

Reporting jobs can use additional resources when they are available.

Machine Learning Queue

Machine learning jobs can require large amounts of CPU and memory.

A dedicated ML queue prevents these jobs from interfering with ETL and reporting.

When the cluster has unused capacity, ML workloads can use additional resources.

Fairness

Fairness is achieved by:

- Giving each queue a guaranteed capacity.
- Preventing one workload from consuming all cluster resources.
- Allowing unused resources to be temporarily shared.
- Using controlled preemption when necessary.
- Applying resource limits to individual applications.

For example, if the ML queue is using 25% of its guaranteed capacity and ETL suddenly requires more resources, the scheduler can allocate available resources to ETL.

ETL Priority

ETL should have higher priority because it may be part of a critical data pipeline.

Priority can be implemented through:

- A high-priority ETL queue.
- Guaranteed minimum capacity.
- Controlled resource preemption.

Preemption should be used carefully because repeatedly stopping and restarting tasks can reduce overall performance.

Node Failure Recovery

YARN consists of:

- ResourceManager.
- NodeManagers.
- ApplicationMasters.

The ResourceManager manages cluster resources.

NodeManagers run containers on worker nodes.

If a worker node fails, the ResourceManager detects the failed NodeManager.

The failed containers are identified, and the ApplicationMaster requests replacement containers on healthy nodes.

The affected tasks are then restarted.

For example:

1. A node fails.
2. ResourceManager detects the failure.
3. Containers running on that node are considered failed.
4. ApplicationMaster requests new containers.
5. YARN allocates containers on healthy nodes.
6. Failed tasks are restarted.

This provides recovery without requiring the entire application to restart.

Final Policy

The scheduler should therefore use:

- Capacity Scheduler.
- Separate queues for ETL, Reporting and ML.
- High priority for ETL.

- Guaranteed capacity for all workloads.
- Controlled sharing of unused resources.
- Controlled preemption.
- YARN automatic task recovery.

4. Enterprise Data Ingestion from SAN, NAS and Operational Databases

Introduction

An enterprise may have data stored in:

- SAN systems.
- NAS systems.
- Operational databases.

This data needs to be transferred into a big-data platform while ensuring:

- Reliable backup.
- Minimal duplication.
- Controlled data movement.
- Centralized processing.

Proposed Architecture

The architecture should contain the following major components:

1. Source systems.
2. Ingestion layer.
3. Raw data storage.
4. Processing layer.
5. Curated storage.
6. Backup storage.
7. Metadata management.

SAN Integration

SAN provides block-level storage and may contain enterprise application data.

Only the required datasets should be extracted from SAN and transferred to the big-data platform.

The system should avoid repeatedly copying the entire SAN dataset.

Where appropriate, data can be exported or exposed through a controlled ingestion mechanism.

NAS Integration

NAS provides file-level storage.

It may contain:

- Documents.
- Images.
- Shared files.
- Historical datasets.

The ingestion layer can monitor required NAS locations and transfer new or changed files into the Hadoop data lake.

Operational Database Integration

Operational databases contain structured information such as:

- Customers.
- Orders.
- Transactions.
- Employees.

Database ingestion should use **incremental extraction or Change Data Capture (CDC)** instead of repeatedly copying the entire database.

For example, if 1 million records exist but only 5,000 changed since the previous ingestion, only those changed records should be transferred.

This reduces:

- Network usage.
- Processing time.
- Storage duplication.
- Database load.

Apache NiFi as Ingestion Layer

Apache NiFi is suitable as the centralized ingestion and flow-management layer.

It can:

- Connect to different sources.
- Route data.

- Validate records.
- Transform data.
- Monitor data flows.
- Maintain provenance information.
- Apply back pressure.
- Transfer data to HDFS.

This gives the enterprise a single controlled ingestion layer.

Raw Data Zone

Data should first be stored in a Raw Zone.

The Raw Zone preserves the original source data.

This is useful because the original data can be reprocessed later if business rules change.

Processing Layer

After ingestion, Spark, MapReduce, or ETL tools can clean and transform the data.

For example:

- Remove invalid records.
- Standardize formats.
- Remove unwanted fields.
- Join datasets.
- Apply business rules.

The processed data is then stored separately.

Minimal Duplication

Minimal duplication can be achieved through:

Incremental Ingestion

Only new or changed data is transferred.

Change Data Capture

Database changes are captured rather than copying complete tables.

Metadata Tracking

Metadata should maintain:

- Source name.

- Dataset name.
- Ingestion time.
- Last processed timestamp.
- File or record version.
- Target location.

Unique Identifiers

Records should have stable identifiers to detect duplicates.

For files, checksums or hashes can be used to determine whether a file has already been ingested.

Backup Reliability

HDFS replication protects against individual hardware failures, but replication alone should not be considered a complete backup strategy.

The architecture should have:

- HDFS replication for operational fault tolerance.
- Separate backup storage for recovery from accidental deletion or corruption.
- Periodic backup verification.
- Appropriate retention policies.

Critical data should have backup copies in a separate failure domain whenever possible.

Final Architecture Components

Component	Purpose
SAN	Source for block-level enterprise data
NAS	Source for file-based data
Operational DB	Source for structured business data
Apache NiFi	Data ingestion and flow management
HDFS	Distributed big-data storage
Spark/MapReduce	Data processing
Metadata Catalog	Track datasets and ingestion status
Backup Storage	Disaster and recovery protection
Hive/BI tools	Analytics and reporting

5. End-to-End Hadoop Ecosystem with Secure Data Movement and High Availability

Introduction

An enterprise Hadoop ecosystem must support:

- Secure movement of data.
- High availability.
- Integration with Apache NiFi or ETL pipelines.
- Large-scale storage.
- Distributed processing.
- Analytics.

A suitable solution combines **Apache NiFi, HDFS, YARN, Spark/MapReduce, Hive and Hadoop security mechanisms.**

1. Data Sources

The ecosystem can receive data from:

- Operational databases.
- Application logs.
- Files.
- APIs.
- IoT systems.
- SAN.
- NAS.

These sources send data to the ingestion layer.

2. Apache NiFi Ingestion Layer

Apache NiFi acts as the main data-flow management system.

A typical process is:

1. Receive data from the source.
2. Validate the data.
3. Identify the data type.
4. Apply required transformations.
5. Route the data.

6. Encrypt or securely transfer it.

7. Store it in HDFS.

NiFi is useful because it provides flow monitoring, routing, back pressure, provenance and integration with many different systems.

3. Secure Data Movement

Security should be applied while data is moving between systems.

Encryption

Use TLS/HTTPS or other secure transport mechanisms to protect data in transit.

This prevents unauthorized users from reading data while it travels through the network.

Authentication

Hadoop services can use Kerberos-based authentication in an enterprise environment.

Only authenticated users and services should be allowed to access protected resources.

Authorization

Access controls should define who can access specific datasets.

For example, sensitive employee data should only be accessible to authorized users and applications.

4. HDFS Storage

After ingestion, data is stored in HDFS.

The data can be organized into different zones:

Raw Zone

Stores data in its original form.

Processed Zone

Contains cleaned and transformed data.

Curated Zone

Contains business-ready data used for analytics and reporting.

This organization improves data management and makes the pipeline easier to maintain.

5. HDFS High Availability

The NameNode should use a High Availability configuration consisting of:

- Active NameNode.
- Standby NameNode.

- JournalNodes.
- Automatic failover.

If the Active NameNode fails, the Standby NameNode becomes active.

This prevents a NameNode failure from making the entire HDFS unavailable.

6. DataNode Fault Tolerance

HDFS should use replication to protect data from DataNode failures.

A replication factor of 3 is commonly used where the storage budget permits.

If one DataNode fails, another replica can serve the data.

HDFS can also create a replacement replica to restore the desired replication level after a failure.

7. YARN Resource Management

YARN manages resources required by processing applications.

It consists primarily of:

- ResourceManager.
- NodeManagers.
- ApplicationMasters.

YARN allocates CPU and memory to applications and allows multiple workloads to run on the same Hadoop cluster.

8. Processing Layer

Spark

Spark can be used for:

- ETL.
- Large-scale transformations.
- Machine learning.
- Analytics.

MapReduce

MapReduce can be used for:

- Large batch-processing workloads.
- Traditional Hadoop processing.

Both can run under YARN.

9. Hive

Hive provides SQL-based access to Hadoop data.

For example, analysts can query processed data without directly writing MapReduce programs.

Hive is therefore useful for:

- Reporting.
- Data analysis.
- Business intelligence.
- Aggregation.

10. ETL Integration

The overall ETL flow can be:

1. Data is collected from source systems.
2. NiFi receives and validates it.
3. NiFi securely transfers it to HDFS.
4. Raw data is stored in the Raw Zone.
5. Spark or another ETL engine cleans and transforms it.
6. Processed data is stored in HDFS.
7. Hive provides SQL access.
8. BI applications consume the curated data.

11. Failure Recovery

The architecture should support recovery at multiple levels.

NameNode Failure

The Standby NameNode takes over.

DataNode Failure

HDFS serves data from another replica and creates replacement replicas.

YARN Container Failure

YARN can restart failed tasks or containers on healthy nodes.

NiFi Node Failure

A NiFi cluster can provide redundancy so that data-flow processing does not depend on one NiFi instance.

12. Security and Availability Summary

Requirement	Solution
Secure data movement	TLS/HTTPS and secure transfer protocols
Authentication	Kerberos and service authentication
Authorization	HDFS permissions and enterprise access controls
NameNode availability	Active/Standby NameNodes
DataNode fault tolerance	HDFS replication
Processing recovery	YARN container/task recovery
Data ingestion	Apache NiFi
ETL	Spark/ETL pipelines
Batch processing	MapReduce/Spark
SQL analytics	Hive
Data organization	Raw, Processed and Curated zones